Supplementary Materials for "Persistent Dealer-Gamma Regimes Under Temporal Obfuscation: A Re-Run on Complete Inputs with a Controlled LLM Re-Test"

Christopher Regan and Ying Xie

S1 Series Definitions in SQL

The daily series of Table 1 of the article are defined by the four read-only statements of Sections S1.1 to S1.4, which were run on the project database on 2026-10-28; Section S1.5 defines the derived and recovered series. In the statements `:sym` is the symbol (SPY, QQQ or IWM) and `:y0`, `:y1` bound one calendar year, because the raw table is read one symbol-year per statement. Every statement stops at 2026-10-09, the last day before the vendor switch (Section 3.3 of the article). The first two statements return the 30 daily values of every window. The pipeline stores the daily rows and cuts the windows in a second step (Section 3.4 of the article); unlike that step, the first statement drops a window when one of its 30 days of the `gex.options_daily_summary` calendar is missing from the archive table.

S1.1 Pre-Rebuild Archive Table (`old_bs`)

```
-- old_bs: pre-rebuild archive table (read-only). Parameter :sym is the symbol.
-- Calendar: gex.options_daily_summary for the symbol; a 30-day window is valid only if
↪ all 30 consecutive calendar days exist in the archive table.
-- The archive ends on 2026-10-02, so the last window ends there; IWM and QQQ have no
↪ rows from 2020-12-03 to 2025-12-30.
WITH cal AS (SELECT trading_date, row_number() OVER (ORDER BY trading_date) AS rn
             FROM gex.options_daily_summary WHERE symbol = :'sym' AND trading_date <=
↪ DATE '2026-10-09'),
     f AS (SELECT c.rn, c.trading_date, z.total_gex, z.net_call_gex, z.net_put_gex, z.
↪ underlying_price
           FROM cal c JOIN backups.options_daily_summary_pre_rebuild_20261004 z
             ON z.symbol = :'sym' AND z.trading_date = c.trading_date)
SELECT e.trading_date AS window_end,
       array_agg(w.total_gex     ORDER BY w.rn) AS total_gex_30,
       array_agg(w.net_call_gex  ORDER BY w.rn) AS call_gex_30,
       array_agg(w.net_put_gex   ORDER BY w.rn) AS put_gex_30
FROM f e JOIN f w ON w.rn BETWEEN e.rn - 29 AND e.rn
GROUP BY e.trading_date, e.rn
HAVING count(*) = 30          -- hole-free window
ORDER BY e.rn;
```

S1.2 Rebuilt Daily Table (`new_full`)

```
-- new_full: stored per-contract gamma, same-day expiries included (gex.
↪ options_daily_summary, rebuilt 2026-10-08). Parameter :sym is the symbol.
WITH s AS (SELECT trading_date, total_gex, net_call_gex, net_put_gex, underlying_price,
                  row_number() OVER (ORDER BY trading_date) AS rn
           FROM gex.options_daily_summary
           WHERE symbol = :'sym' AND trading_date <= DATE '2026-10-09')      --
↪ alpha_vantage ends 2026-10-09; cboe_delayed starts 2026-10-12
SELECT e.trading_date AS window_end,
       array_agg(w.total_gex    ORDER BY w.rn) AS total_gex_30,
```

```
       array_agg(w.net_call_gex ORDER BY w.rn) AS call_gex_30,
       array_agg(w.net_put_gex  ORDER BY w.rn) AS put_gex_30
FROM s e JOIN s w ON w.rn BETWEEN e.rn - 29 AND e.rn
WHERE e.rn >= 30
GROUP BY e.trading_date, e.rn
HAVING count(*) = 30
ORDER BY e.rn;
```

S1.3 Raw Chains Without Same-Day Expiries (`new_ex0dte`)

```
-- new_ex0dte: raw chains, one symbol-year per statement. Parameters :sym, :y0, :y1 (SPY
↪  and IWM 2008 to 2026, QQQ 2011 to 2026).
-- enable_seqscan=off is a session planner setting (no writes): the index plan is faster
↪  and returns the same numbers (largest relative difference 5e-13).
SET enable_seqscan = off;
SELECT trading_date,
       count(*) FILTER (WHERE expiration > trading_date)        AS n_contracts,
       count(*) FILTER (WHERE expiration = trading_date)        AS n_0dte,
       sum(CASE WHEN option_type = 'call' THEN g END)  FILTER (WHERE expiration >
↪ trading_date) AS call_gex_ex0dte,
      -sum(CASE WHEN option_type = 'put'  THEN g END)  FILTER (WHERE expiration >
↪ trading_date) AS put_gex_ex0dte,
       sum(CASE WHEN option_type = 'call' THEN g ELSE -g END) FILTER (WHERE expiration >
↪  trading_date) AS total_gex_ex0dte,
       sum(CASE WHEN option_type = 'call' THEN g ELSE -g END) FILTER (WHERE expiration =
↪  trading_date) AS total_gex_0dte,
       sum(CASE WHEN option_type = 'call' THEN g ELSE -g END)
↪                AS total_gex_all  -- ties to gex.options_daily_summary.total_gex (rel.
↪  diff. < 3e-7)
FROM (SELECT trading_date, option_type, expiration,
             gamma::float8 * open_interest::float8 * 100.0 * underlying_price::float8 *
↪ underlying_price::float8 * 0.01 AS g   -- float8: real*int would stay float4
      FROM gex.options_chains_partitioned
      WHERE symbol = :'sym' AND trading_date >= :'y0' AND trading_date < :'y1'
        AND trading_date <= DATE '2026-10-09'                      -- vendor cap:
↪ alpha_vantage ends 2026-10-09, cboe_delayed starts 2026-10-12
        AND gamma IS NOT NULL AND open_interest > 0 AND underlying_price > 0) t
GROUP BY trading_date
ORDER BY trading_date;
```

Since the database additions of 2026-11-03 (Section 3.3 of the article) the pipeline's statement also takes the spot from `gex.spot_fill_log` on the days listed there, whose chain rows have no spot: it reads `coalesce(c.underlying_price, f.spot_used)` from a left join of the chains with that table on symbol and trading date (rows with `in_summary`), and keeps the row filter otherwise. On every other day the result is unchanged.

S1.4 Late-Session Intraday Snapshot (`intraday_late`)

The statement as listed returns Variant 1 of its comments, the `market_close` slot, which equals the end-of-day row. The series of the study is Variant 2: the filter on the clock time is replaced by a filter on the snapshot type (`snapshot_type` equal to `final_rush`, 19:55:59 UTC), so that the snapshot is selected by type and not by clock time, and the three irregular days 2026-07-29, 2026-08-03 and 2026-09-14 are dropped when the series is built.

```
-- intraday_late: late-session snapshot, SPY, 2026-07-27 to 2026-10-09 (EDT: 20:00 UTC =
↪  16:00 ET).
-- Variant 1 'late' = last snapshot with timestamp < 20:01:00 UTC  -> the market_close
↪ slot (20:00:59), which EQUALS the daily summary row (not independent).
-- Variant 2, used in this study (independent of the end-of-day row): add  AND
↪ snapshot_type = 'final_rush'  (19:55:59 UTC).
SELECT DISTINCT ON (d)
       d AS trading_date, snapshot_timestamp, snapshot_type, total_gex, net_call_gex,
↪ net_put_gex, underlying_price, total_contracts
FROM (SELECT *, (snapshot_timestamp AT TIME ZONE 'America/New_York')::date AS d
      FROM gex.intraday_gex_summary
      WHERE symbol = 'SPY'
```

```
        AND snapshot_timestamp >= TIMESTAMPTZ '2026-07-27 00:00+00' AND
↪ snapshot_timestamp < TIMESTAMPTZ '2026-10-10 00:00+00'
        AND snapshot_type <> 'test'
        AND (snapshot_timestamp AT TIME ZONE 'UTC')::time < TIME '20:01:00') x
ORDER BY d, snapshot_timestamp DESC;
```

S1.5 Derived and Recovered Series

Call leg only (`call_only`). The sum of the call terms g of the third statement over all expiries, same-day expiries included, with the put terms dropped. In the rebuilt summary table this is `net_call_gex` (up to the float4 rounding of the stored column), which the second statement returns as `call_gex_30`.

Scaled series (`new_full_scaled`). The `new_full` values of QQQ and IWM multiplied by one fixed factor per symbol (Section 3.6 of the article), which the following statement computes (tested read-only on 2026-10-29). The factor and the number of days are stored with the series.

```
-- new_full_scaled: factor of one symbol = median over 2020-01-01..2026-10-09 of the
↪ ratio of SPY's open-interest notional to the symbol's.
-- Parameter :sym is QQQ or IWM; the scaled values are the new_full values of the symbol
↪  times this factor.
WITH n AS (SELECT symbol, trading_date, total_weighted_oi::float8 * 100.0 *
↪ underlying_price::float8 AS notional
          FROM gex.options_daily_summary
          WHERE symbol IN ('SPY', :'sym') AND trading_date BETWEEN DATE '2020-01-01'
↪ AND DATE '2026-10-09')
SELECT percentile_cont(0.5) WITHIN GROUP (ORDER BY s.notional / q.notional) AS factor,
       count(*) AS n_days
FROM n s JOIN n q ON q.trading_date = s.trading_date AND s.symbol = 'SPY' AND q.symbol =
↪  :'sym'
WHERE q.notional > 0;
```

Recovered input series. The series `jan2026_phase4a`, `p3_2024` and `p4_2020` (and the dated twins of the last two) are not defined by SQL. They are the daily values parsed from the data lines of the surviving batch input files `phase4a_postgresql_20260104_184832.jsonl`, `phase3_full_2024_baseline_v2.jsonl` and `batch_regime_20251120_104630.jsonl` (folder `reports/validation/paper2_regime_windows/batch_jobs/`); overlapping windows are checked for consistent values (Section 4.1 of the article).

S2 Run Ledger

Tables S1 and S2 list every model run of the study: the identifiers, the token usage and the cost. The arms are defined in Section 3.6 of the article.

Table S1: Run ledger: one row per Batch job, with the token usage and the cost priced at Batch rates. The identifier has the form `<arm>_<model>_<mode>_<UTC timestamp>_c<chunk>`. Only runs paid in this study enter the total; the synchronous pilot (`sync`) is included in it. The eight jobs with the timestamp 20261103 are the re-requests of windows whose input changed after the database additions of 2026-11-03 (Section 3.7 of the article): 1,049 of their 1,121 answers replace answers of the earlier jobs, which stay in the ledger and in the total. The three backfilled o4-mini runs are listed below it, as stored answers of earlier runs that are not spend of this study.

Run identifier	Requests	Prompt tokens	Completion tokens	Cost (USD)	Status
bridge_jan_gpt-6-luna_batch_20261029T203742Z_c0	1,476	2,823,588	1,086,712	0.41	ingested
bridge_jan_gpt-6-luna_batch_20261029T203742Z_c1	1,476	2,823,588	1,082,280	0.41	ingested
full_spy_gpt-6-luna_batch_20261029T203745Z_c0	1,227	2,347,251	871,277	0.34	ingested
full_spy_gpt-6-luna_batch_20261029T203745Z_c1	1,226	2,345,338	969,842	0.36	ingested
full_spy_gpt-6-luna_batch_20261029T203745Z_c2	1,226	2,345,338	1,004,850	0.37	ingested
full_spy_gpt-6-luna_batch_20261029T203745Z_c3	1,226	2,345,338	1,024,540	0.37	ingested
full_spy_gpt-6-luna_batch_20261103T040042Z_c0	348	665,724	272,121	0.10	ingested
ex0dte_spy_gpt-6-luna_batch_20261029T203749Z_c0	1,227	2,347,251	839,753	0.33	ingested
ex0dte_spy_gpt-6-luna_batch_20261029T203749Z_c1	1,226	2,345,338	968,965	0.36	ingested
ex0dte_spy_gpt-6-luna_batch_20261029T203749Z_c2	1,226	2,345,338	951,505	0.36	ingested
ex0dte_spy_gpt-6-luna_batch_20261029T203749Z_c3	1,226	2,345,338	982,560	0.36	ingested
ex0dte_spy_gpt-6-luna_batch_20261103T040045Z_c0	332	635,116	263,091	0.10	ingested
p34_obf_2024_gpt-6-luna_batch_20261029T203752Z_c0	234	447,642	156,636	0.06	ingested
p34_dated_2024_gpt-6-luna_batch_20261029T203754Z_c0	234	461,682	154,330	0.06	ingested
p34_obf_2020_gpt-6-luna_batch_20261029T203757Z_c0	234	447,642	132,619	0.06	ingested
p34_dated_2020_gpt-6-luna_batch_20261029T203759Z_c0	234	461,682	130,805	0.06	ingested
full_qqq_gpt-6-luna_batch_20261029T204029Z_c0	1,354	2,590,202	1,012,074	0.38	ingested
full_qqq_gpt-6-luna_batch_20261029T204029Z_c1	1,353	2,588,289	1,004,705	0.38	ingested
full_qqq_gpt-6-luna_batch_20261029T204029Z_c2	1,353	2,588,289	1,057,334	0.39	ingested
full_qqq_gpt-6-luna_batch_20261103T040049Z_c0	134	256,342	107,963	0.04	ingested
full_iwm_gpt-6-luna_batch_20261029T204033Z_c0	1,228	2,349,164	762,229	0.31	ingested

Table S2: Run ledger (continued): remaining Batch jobs, the total of this study and the backfilled runs. Rows are rounded to cents; the totals are computed from the unrounded values (the three backfilled runs sum to 9.97).

Run identifier	Requests	Prompt tokens	Completion tokens	Cost (USD)	Status
full_iwm_gpt-6-luna_batch_20261029T204033Z_c1	1,228	2,349,164	719,759	0.30	ingested
full_iwm_gpt-6-luna_batch_20261029T204033Z_c2	1,228	2,349,164	826,909	0.32	ingested
full_iwm_gpt-6-luna_batch_20261029T204033Z_c3	1,228	2,349,164	855,981	0.33	ingested
full_iwm_gpt-6-luna_batch_20261103T040052Z_c0	184	351,992	113,134	0.05	ingested
full_qqq_scaled_gpt-6-luna_batch_20261029T204036Z_c1	877	1,677,701	750,362	0.27	ingested
full_qqq_scaled_gpt-6-luna_batch_20261029T204036Z_c0	877	1,677,701	694,967	0.26	ingested
full_qqq_scaled_gpt-6-luna_batch_20261103T040056Z_c0	31	59,303	28,589	0.01	ingested
full_iwm_scaled_gpt-6-luna_batch_20261029T204039Z_c0	878	1,679,614	614,557	0.24	ingested
full_iwm_scaled_gpt-6-luna_batch_20261029T204039Z_c1	877	1,677,701	695,367	0.26	ingested
full_iwm_scaled_gpt-6-luna_batch_20261103T040059Z_c0	30	57,390	26,336	0.01	ingested
ex0dte_qqq_gpt-6-luna_batch_20261030T010447Z_c1	892	1,706,396	680,360	0.26	ingested
ex0dte_qqq_gpt-6-luna_batch_20261030T010447Z_c0	892	1,706,396	622,593	0.24	ingested
ex0dte_qqq_gpt-6-luna_batch_20261103T040102Z_c0	30	57,390	20,013	0.01	ingested
ex0dte_iwm_gpt-6-luna_batch_20261030T010451Z_c0	893	1,708,309	560,829	0.23	ingested
ex0dte_iwm_gpt-6-luna_batch_20261030T010451Z_c1	892	1,706,396	616,040	0.24	ingested
ex0dte_iwm_gpt-6-luna_batch_20261103T040105Z_c0	32	61,216	20,550	0.01	ingested
intraday_spy_gpt-6-luna_batch_20261029T204042Z_c0	54	103,302	45,525	0.02	ingested
bridge_old_gpt-6-luna_batch_20261030T010455Z_c0	1,218	2,330,034	813,090	0.32	ingested
bridge_old_gpt-6-luna_batch_20261030T010455Z_c1	1,218	2,330,034	970,974	0.36	ingested
bridge_old_gpt-6-luna_batch_20261030T010455Z_c2	1,217	2,328,121	953,927	0.35	ingested
bridge_old_gpt-6-luna_batch_20261030T010455Z_c3	1,217	2,328,121	830,288	0.32	ingested
pilot_bridge_jan_gpt-6-luna_sync_20261029T203454Z_c0	40	76,520	28,015	0.02	ingested
Total (this study)	35,833	68,576,609	26,324,356	10.02	
Backfilled o4-mini runs (not spend of this study)					
r1_o4mini_phase4a_20260104	1,476	2,655,324	2,928,992	7.90	backfill
r1_p3_o4mini_20251120	223	414,557	405,127	1.12	backfill
r1_p4_o4mini_20251120	223	414,557	325,436	0.94	backfill